# Moving to a more rigorous development process

**Hermes looks like a good fit for the agent framework layer**, because your core problem was: *operational knowledge improves through use and needs to become reusable procedure*. Hermes explicitly separates **memory = facts** from **skills = procedures**, and its skills are on-demand reusable documents the agent can create/modify. (Hermes Agent)

The architecture would be:

Temporal = durable workflow state
OB1 = evidence, profile, application history, semantic memory
Hermes = evolving skills / agent behavior
App repo = versioned code, schemas, adapters, tests

As an example, a job application:

Temporal workflow: CreateApplication

1. Ingest JD
2. Ask Hermes: evaluate JD using jd-evaluation skill
3. Store JD, score, rationale in OB1
4. Ask Hermes: generate resume using resume-generation skill
5. Verify claims against OB1 evidence
6. Generate cover letter / portal answers
7. Update tracker
8. Capture user corrections
9. Propose skill updates

The crucial evolution loop:

Run workflow
↓
User corrects output
↓
Correction stored in OB1 as event/evidence
↓
Hermes updates relevant skill
↓
Regression test runs against prior JDs
↓
If tests pass, promote skill version

So instead of a vague Claude memory note like:

“Use company descriptions in resumes”

you get a versioned skill change:

skills/resume-generation/SKILL.md
Rule:
When listing employers that may be obscure to recruiters, include a short company descriptor unless the brand is broadly recognizable.

Examples:
- LatticeFlow AI — AI risk management platform...
- Drawing Management — geospatial document intelligence...

And instead of scattered OB1/local rules, you’d have:

storage-routing.skill.md
Use StorageAdapter.
Never call local file paths directly from workflow logic.
Backend selected by config:
- local
- ob1

Temporal matters because job search is not a single chat. It is a long-running process with pauses, retries, state transitions, and follow-ups. Temporal describes its workflow execution as durable, reliable, and scalable, and its platform is designed to resume execution after failures. (Temporal Docs)

So my answer is:

**Hermes as agent framework, yes. But not alone.**
Hermes should own evolving procedural knowledge. Temporal should own workflow state. OB1 should own durable applicant/job/evidence memory. Claude Code should remain the coding tool, not the operational brain.

---

Architecture:

Webapp
↓
Workflow API
↓

Agent Runtime (Hermes)
↓
Versioned Skills
↓
Tools
↓
OB1

So Hermes sits between the workflow layer and the skill layer.

Something like:

POST /create-application

arrives.

The workflow service decides:

```
workflow:
  - evaluate-jd
  - generate-resume
  - generate-cover-letter
  - update-tracker
```

Hermes then executes:

```
evaluate-jd
   uses:
      jd-evaluation-v3

generate-resume
   uses:
      resume-generation-v7

generate-cover-letter
   uses:
      cover-letter-v2
```

The distinction is subtle but important.

---

## What should be first-class modules?

The things you've been learning in Claude Code.

Examples from your repo:

memory/
CLAUDE.md
workflow.md
resume-generation-rules
OB1 routing rules
company description rules

These become:

skills/
  jd-evaluation
  resume-generation
  cover-letter

policies/
  factuality
  evidence-grounding
  company-descriptors
  storage-routing

workflows/
  create-application
  interview-prep

---

## What is Hermes actually doing?

Hermes is not the skill.

Hermes is the executor.

Think:

Skill
  =
  procedure

Hermes
=
worker that executes procedure

Exactly like:

Python file
=
code

Python interpreter
=
executor

---

## But here's where Hermes becomes interesting

The learning loop.

Suppose recruiters (or you) repeatedly correct resumes:

Add company descriptions

Hermes observes:

Resume outputs
Resume edits
Final accepted resumes

and proposes:

resume-generation-v8

That proposal gets committed into:

skills/resume-generation/v8.md

Then production starts using v8.

This is why I think Hermes is potentially valuable.

Not because it runs skills.

Many frameworks can do that.

Because it can help evolve them.

---

## If I were refactoring your job-search repo

Looking at what you've built, I'd probably evolve it to:

job-search

skills/
  jd-evaluation/
  resume-generation/
  cover-letter/
  interview-prep/

policies/
  factuality/
  evidence-grounding/
  ob1-routing/

workflows/
  create-application/
  prepare-interview/

services/
  ob1/
  tracker/
  documents/

webapp/

Then:

Webapp
 ↓
Workflow API
 ↓

Hermes Runtime
↓
Skills
↓
OB1 / Services

---

For **job-search**, Hermes can plausibly be both:

Runtime
+
Learning engine

But for the job-search application, where there is only one tenant (you), using Hermes as both the executor and the skill-learning system is completely reasonable and probably the fastest path to proving the architecture.

---

You want **one Hermes-centered runtime**, with two entry points:

Direct interactive use

→ Hermes Runtime

→ Versioned Skills

→ Tools

→ OB1

Webapp use

→ Workflow API

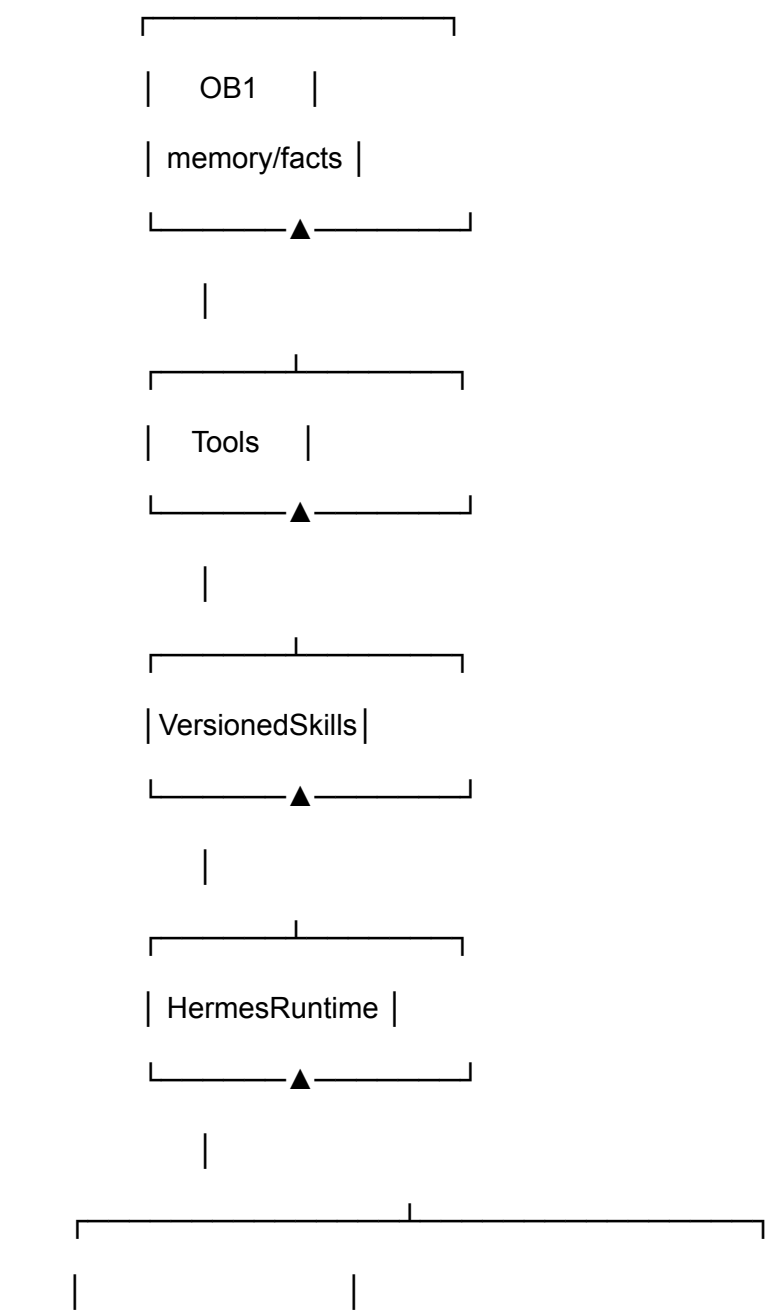
→ Hermes Runtime

→ Versioned Skills

→ Tools

→ OB1

So the architecture becomes:



That preserves what worked: **you refined rules while doing real work**.

The key design distinction should be:

**interactive mode may propose/edit skills**
**webapp mode should execute pinned skill versions by default**

Example:

Interactive Hermes:

resume-generation@draft

Webapp:

resume-generation@v7

Then when the interactive refinement proves better:

draft → tests → v8 → webapp uses v8

So yes: put Hermes in the middle. But add **skill versioning and execution modes**:

mode: interactive

can use draft skills

can propose changes

can compare versions

mode: webapp

uses approved versions

logs feedback/corrections

may request refinement

That gives you both: live iteration and stable operational execution.

---

**Hermes can coordinate OB1 changes**, but it should not directly mutate OB1 without guardrails.

Use this pattern:

Hermes Runtime
→ plans the change
→ calls OB1 tools/API
→ records event/audit trail
→ validates result
→ updates skill if needed

For example:

User correction:
"Add CA Technologies, JPMC Wealth, and Raymond James as named Jaspersoft customers."

Hermes coordinates:
1. classify as profile/evidence update
2. create OB1 event
3. update relevant employer/customer facts
4. link facts to source note
5. rerun resume-generation skill
6. record that resume-generation-v8 depends on those facts

But I'd separate OB1 changes into two classes:

Normal data updates
- new job
- new JD
- interview notes
- application status
- profile correction
- source evidence

→ Hermes can write through approved OB1 tools

Schema / ontology updates
- new entity type
- new relationship type
- new indexing rule
- new storage policy

→ Hermes proposes; Claude Code / migration handles

So:

Hermes can coordinate content and workflow-memory updates.

Hermes should propose structural OB1 changes, not apply them directly.

The useful architecture is:

Hermes
↓
OB1 Change Planner
↓
OB1 Tool Layer
↓
Validation + Audit
↓
OB1

And every OB1 write should include:

who/what caused it
source evidence
timestamp
workflow/application context
skill version involved
confidence / review status

That way OB1 remains the trusted memory layer, not just another agent scratchpad.